ФАХОВИЙ КОЛЕДЖ
МНТУ

Об'єктно орієнтоване програмування

Практичне заняття №7. ЗМЗ. ЛЗ8.

Тищенко Сергій, alk-43

2026-03-22

1 Аналіз роботи базового і похідного класів у C++

Практичне заняття №7. ЗМЗ. Л38.

1.1 Мета

Розробка та реалізація ієрархії класів мовою C++ із використанням наслідування і віртуальних функцій, організації обчислення факторіала для окремого числа та елементів масиву, а також дослідження особливостей роботи методів базового класу при зверненні через вказівники на базовий клас.

1.2 Класи, наслідування, віртуальні методи, динамічний поліморфізм

У роботі використано два класи:

- базовий клас `Chislo`
- похідний клас `Matrix`

Наслідування реалізовано як:

```
class Matrix : public Chislo
```

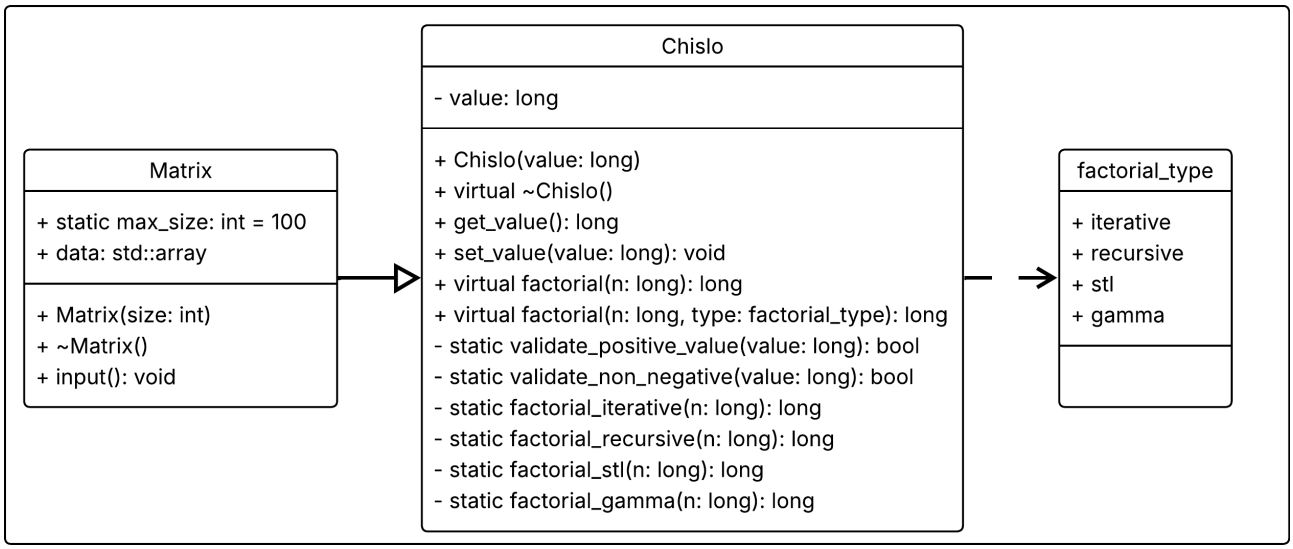

Рис. 1 Діаграма класів.

Це означає, що `Matrix` отримує доступ до публічних та захищених членів `Chislo`.

1.2.1 Віртуальні методи

У базовому класі визначено:

```
virtual long factorial(long n) const;
```

Це дозволяє викликати метод через вказівник на базовий клас:

```
Chislo *chislo = new Matrix;
chislo->factorial(10);
```

1.2.2 Динамічний поліморфізм

Динамічний поліморфізм означає, що виклик функції визначається під час виконання програми, а не компіляції.

У роботі продемонстровано принцип підстановки, коли об'єкт має тип `Matrix` а використовується через `Chislo*`, тобто об'єкт похідного класу може використовуватись як об'єкт базового класу.

1.3 Модифікатори доступу, конструктори, деструктори

Модифікатор	Доступ
private	тільки всередині класу
protected	клас + похідні
public	доступ звідусіль

У класі `Chislo`:

```
private:
  long value_;
```

Інваріант захищений від прямого доступу.

Конструктор

```
Chislo::Chislo(const long value)
{
    validate_positive_value(value);
}
```

Гарантує коректний стан об'єкта.

Деструктор

```
virtual ~Chislo();
```

Необхідний для коректного видалення через базовий вказівник:

```
Chislo* ptr = new Matrix(...);
delete ptr;
```

1.3.1 Інваріанти, перевірки, обробка помилок

Інваріанти `value_ > 0` елементи масиву `>= 0`

Валідація та обробка виключень

```
void validate_positive_value(long value);
void validate_non_negative(long n);

throw std::invalid_argument("...");

try
{
    f();
}
catch (const std::exception& e)
{
    std::cerr << e.what();
}
```

1.4 Шаблони

Продемонстровано використання шаблонної функції на прикладі печаті результатів обчислення факторіала різних реалізацій. Це приклад параметричного поліморфізму:

```
template <typename M, typename C>
void print_type_factorial(M matrix, C chislo)
```

1.5 Вибір функції та обгортка

Вибір реалізації здійснюється через `enum`:

```
enum class factorial_type
{
    iterative,
    recursive,
    stl,
    gamma
};

// Використання:
switch (type)
```

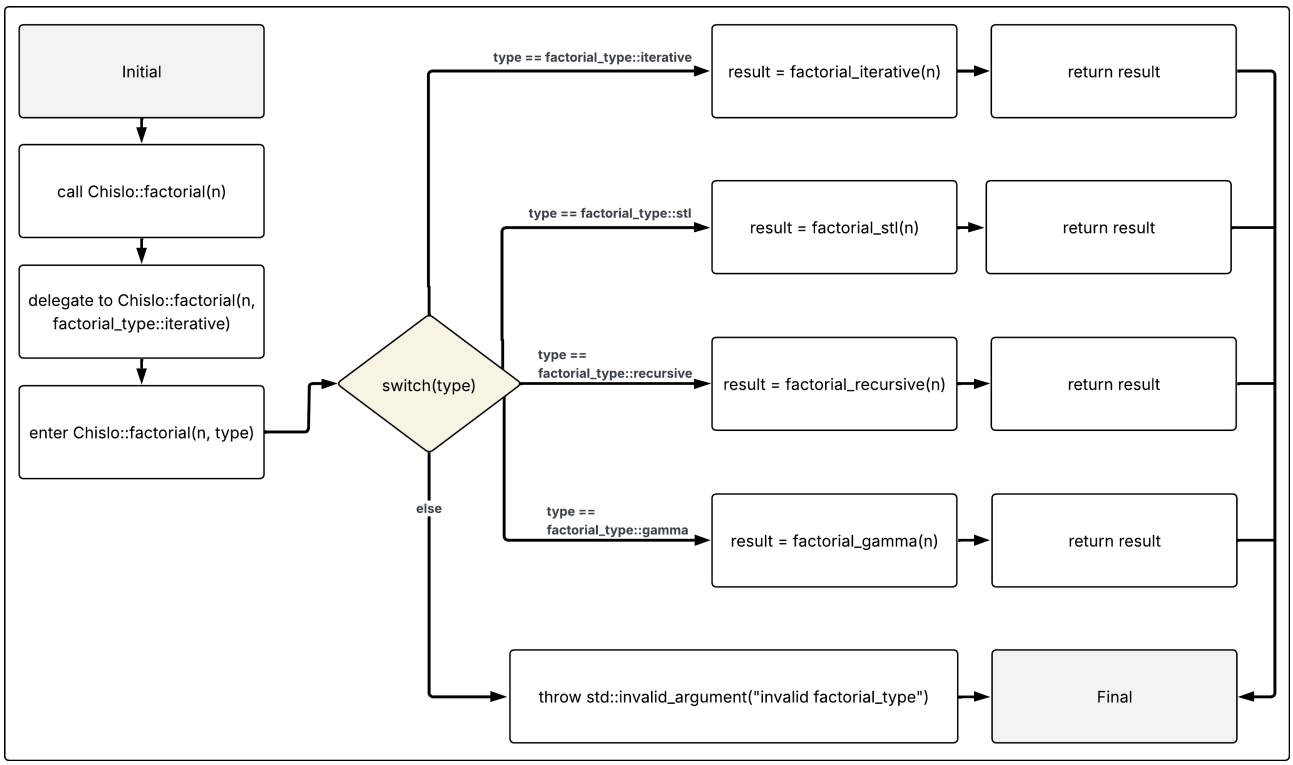

Рис. 2 Блок-схема вибору функції обчислення.

Перевантажена обгортка дозволяє спростити виклик, уникнути дублювання коду, задати значення за замовчуванням а також забезпечує можливість зміни реалізації без зміни клієнтського коду:

```
long factorial(long n) const
{
    return factorial(n, factorial_type::iterative);
}
```

1.6 Методи обчислення факторіала

1.6.1 Ітеративний

$$n! = \prod_{i=1}^{n} i$$

```
for (long i = 2; i <= n; ++i)
    result *= i;
```

1.6.2 Рекурсивний

$$n! = n \cdot (n-1)!$$

```
return n * factorial_recursive(n - 1);
```

1.6.3 STL

Використання стандартних алгоритмів:

```
std::iota(...)
std::accumulate(...)
```

1.6.4 Через гамма-функцію

$$n! = \Gamma(n+1)$$

```
std::tgamma(n + 1)
```

Порівняння методів

Метод	Простота	Швидкість	Надійність
iterative	висока	висока	висока
recursive	середня	нижча	ризик переповнення стеку
stl	висока	середня	залежить від контейнера
gamma	висока	висока	похибка округлення

1.6.5 Виконання

```
Введіть розмір масиву: 5
Введіть обчислювальні елементи: 0 1 2 3 4
Масив:
0 1 2 3 4
Факторіали елементів масиву:
  0!
  iterative: 1
  recursive: 1
  stl: 1
  gamma: 1
-----------------------------------------------------------------
  1!
  iterative: 1
  recursive: 1
  stl: 1
  gamma: 1
-----------------------------------------------------------------
  2!
  iterative: 2
  recursive: 2
  stl: 2
  gamma: 2
-----------------------------------------------------------------
  3!
  iterative: 6
  recursive: 6
  stl: 6
  gamma: 6
-----------------------------------------------------------------
  4!
  iterative: 24
```

Рис. 3 Виведення в консоль.

1.7 Висновок

У роботі реалізовано ієрархію класів із використанням наслідування та віртуальних функцій. Продемонстровано механізм динамічного

поліморфізму через використання вказівників на базовий клас. Забезпечено контроль інваріантів та обробку виключень. Реалізовано декілька алгоритмів обчислення факторіала та проведено їх порівняння. Використання шаблонів дозволило узагальнити функціональність і підвищити гнучкість програмної реалізації.